

RoomKit — Whole-Room AI Design & Commerce Engine

Founder Build Packet (v2) — Business Plan + Claude Code Build Roadmap

Companion to your Build B (Furniture Price Engine) packet. Same operating philosophy and the same conventions you already build with — Supabase + Railway + GitHub, workflow-first architecture, deterministic rules where money is on the line, run-scoped/idempotent jobs, snapshots, staleness guards, comprehensive test suites run green before each stage gate, founder-reviewable artifacts, architecture-first stepwise prompting with exit gates.

What's different from Build B, and why this packet adds surface it didn't have: this product transacts (affiliate money flows on every link), renders (an image pipeline), faces a live buyer (conversion, trust, returns), and has a real frontend (the UI is the funnel). Each of those gets explicit treatment below.

"RoomKit" is a placeholder name.

PART I — BUSINESS PLAN

(Unchanged in substance from v1 — a business plan is a different artifact from a build packet and does not need Build-B build-depth. Condensed here; the operative detail is in Part II.)

1. One-paragraph product

A consumer web app that turns an empty or existing room into a complete, budget-allocated, **shoppable** room. Photo or dimensions + a short style/budget Q&A in; out comes a coherent set of real, buyable products (wall art, bedding, rug, lighting, TV, accents, larger furniture) that match a vibe and **sum to the stated budget**, each with a live buy link, plus an AI render of the styled room. Revenue: affiliate referral in v1, evolving to subscription + sourcing spread when the product turns toward operators.

2. Insight & why the gap exists

Dozens of AI design apps do photo → render. None do "what to buy, what it costs, where to get it, on budget, as a coherent whole." Inspiration is free and crowded; the budgeted shoppable cart is the unowned, monetizable layer. The gap persists because the render is the easy demo-able half and the buy-layer requires real product data, budget math, category-specific sourcing, and willingness to be judged on whether the cart is *good* — the discipline image-first teams skip. That discipline is your edge.

3. Why "whole room" fixes the economics

The furniture-only version's weak number was path-to-\$10k (thin 1–4% affiliate, once-in-years purchase, poor big-ticket conversion). Whole-room scope attacks all three: decor/textiles/accents are **repeat/impulse** purchases (no "already bought my sofa" ceiling), they **convert far better** through affiliate links than a \$1,200 couch, and the category mix is **Amazon-native** (posters, bedding, lighting, rugs, TVs, organization). This moves path-to-\$10k from ~4 toward ~6 and validates Amazon-first.

4. Buyer sequence & the trap

v1 = everyday consumers; destination = operators (STR hosts, stagers, small interior pros). Consumer-first is the only path a solo founder under \$10k can validate fast. The trap: judging the consumer phase by *revenue* (thin affiliate can't hit \$10k at small scale, so you'd wrongly kill a good idea). The correct frame: consumer is a cheap, fast **loop-validation and audience+data engine**; the money is operators + the data layer. **v1 success metric is the loop and click-through, not dollars.**

5. Model & unit economics (honest)

- v1 affiliate-only, ~\$1,200 decor-weighted basket, blended ~3.5%, but credit yourself only a fraction of realized conversion (~\$10–20/converting user). \$10k/mo on affiliate alone ⇒ ~670 converting users/mo ⇒ ~7k–13k engaged sessions/mo. That traffic mountain is *why affiliate is the signal, not the destination*.
- Operator phase: subscription (100 operators × \$49–99 = \$5–10k/mo *predictable*) + **sourcing spread** (trade pricing on home goods — where your pricing-intelligence edge becomes the moat) + affiliate persisting underneath. The financial model **evolves with the buyer**.

6. Durable moat

Not the render (anyone can call an image model) and not the buy-layer (copyable). The only durable asset is the **proprietary furnishing-cost + style-demand + click/conversion data** the engine accumulates. Treat data accumulation as a first-class objective from day one.

7. Competition

Render-first apps (RoomGPT et al. — inspiration only, your funnel competition + differentiation proof); marketplaces (supply, not design — you sit on top); "shop the look" (single item, not a coherent budgeted room); manual staging/STR services (your operator-phase competition, where the weak-competition score lives).

8. GTM (consumer)

Shareable output is the thesis. Organic short-form video (before/after with the shoppable list as payoff), design-obsessed subreddits, and **programmatic SEO** ("furnish a studio for \$X" pages generated by the engine itself — content + product + affiliate surface in one). Every shared board is an ad with your tags embedded.

9. Risks (business)

Crowded render space → lead on the cart; thin/variable affiliate → treat as signal, build to operators; Amazon API gate / Temu no-API → adapter pattern + curated/paid-data fallback; link rot/price drift → deterministic freshness + snapshots; returns/quality (esp. Temu) → Amazon-first for credibility, Temu as opt-in budget tier; poor big-ticket conversion → weight compositions to decor/electronics.

10. Milestones (business)

M1 loop proven (10 users click) · M2 a repeatable traffic channel found · M3 data asset forming · M4 operator pivot readiness (segment with validated WTP → turn on subscription + spread).

PART II — TECHNICAL ARCHITECTURE & BUILD PACKET

Full Build-B-depth. Each phase: what it is · why it matters · what Claude Code should do · artifacts · common founder mistakes · what success looks like · (where relevant) the prompt/code/rule/manual split. Conventions are yours: Supabase + Railway + GitHub, run-scoped idempotent jobs, snapshots, staleness guards, deterministic validators, comprehensive tests green before each gate.

Executive orientation

The simplest correct description: **take a room (photo or dimensions) + style/budget inputs, plan a composition of slots, select real comparable products per slot within budget, snapshot the products/ prices used, validate budget/specs/links/freshness deterministically, render a styled image, and present a budgeted shoppable board — recording every design as a run and every click as an event.** Valuable because it replaces fragmented manual "find things that go together on budget" work and compounds proprietary data. Risky because product data is messy, prices and links go stale between generation and click, taste is subjective, and budget math silently corrupts if it lives in prompts.

Decision	v1 answer
System shape	Workflow-first composition+sourcing pipeline; bounded LLM creativity inside deterministic guardrails
Coding tool	Claude Code in-repo
Frontend	Next.js / React (the UI is the funnel; treated as first-class — Phase 4F)
Backend	Python + FastAPI
DB / state	Supabase (Postgres) — designs, runs, slots, products, snapshots, click events
Deploy	Railway (web + worker + cron), GitHub for source + CI
Core AI use	Style interpretation, composition planning, product selection within constraints, render generation
Deterministic core	Budget allocation/totals, slot/spec requirements, price freshness, link validity, affiliate-tag injection, what the user sees
"Review" analog	No runtime human gate in v1; instead a design-QA eval loop on coherence + budget-edge cases

The Build-B mapping (read this once, it explains every convention below):

Build B (furniture pricing)	RoomKit analog
Item taxonomy (100 targets)	Slot taxonomy (bed, bedding, rug, lamp, wall art, TV, accent...)
Comparability rules / excluded price types	Per-slot spec rules + price-tier rules (bedding needs bed_size; TV needs screen_size; poster needs nothing)
Run-scoped averaging	Run-scoped design generation (each design is a <code>run_id</code> , fully logged, idempotent)

Build B (furniture pricing)	RoomKit analog
Historical snapshots	Product/price snapshot at design time (a saved room is reconstructable even as live prices drift)
Staleness guard before publish	Freshness/staleness guard before a link is shown (and before re-display of an old design)
Advisory locks / deadlock fixes	Locked refresh worker (concurrent price/link refresh must not double-run or deadlock)
Deterministic average computation	Deterministic budget totals + validation (no over-budget board, no dead/untagged link reaches the user)
Anomaly review queue	Design-QA queue for low-coherence / budget-edge designs
316/316 test suite discipline	Same — comprehensive unit/validator/integration tests, green before each stage gate

Phase 1 — Product framing

What it is. Define the product operationally before code, so Claude Code doesn't build the wrong thing elegantly. The correct framing is **not** "AI interior designer" and **not** "render app." It is a **composition-and-sourcing engine** producing budgeted, live, shoppable rooms, with rendering as a presentation layer.

Why it matters. Every later phase — schema design, validator placement, the deterministic/LLM line — depends on whether the product is framed around the *cart* (correct) or the *picture* (wrong). Frame it around the picture and you'll overinvest in render quality and let budget/link logic rot inside prompts.

What Claude Code should do. Draft framing artifacts only — no app code. Restate the product in plain English; write one-paragraph and one-page definitions; separate the genuinely-LLM steps (style interpretation, selection) from the must-be-deterministic steps (budget, specs, links, tags); write the v1 out-of-scope list; write the assumptions-and-risks memo.

Artifacts: `docs/product_overview.md`, `docs/v1_scope.md`, `docs/architecture_choice.md`, `docs/assumptions_and_risks.md`, `schemas/room_taxonomy.schema.json` (first slot-taxonomy draft).

Common founder mistakes: letting the render become the product; budget/affiliate logic in prompts; "the room looks nice" as the success test instead of "coherent + on-budget + clickable + reconstructable"; skipping the slot taxonomy and letting free-form product lists flow between steps.

Success before moving on: you can state the product in one paragraph, list the pipeline stages in order, name where bounded LLM creativity belongs and where it's forbidden, and point to a written out-of-scope list and a draft slot taxonomy.

The prompt/code/rule/manual split at this stage: prompt-driven = drafting memos only; code = not started; rule = the slot taxonomy + spec/price-tier rules begin life as written artifacts; manual = confirm the taxonomy and the v1 scope line.

Phase 2 — System design

What it is. Turn the idea into `input → processing → output → state` and fix the deterministic/LLM line. -

Input: room photo or dimensions; style/budget Q&A; slot taxonomy; sourcing config; affiliate tags. - **Processing:** interpret style → plan composition (slots + per-slot budget) → select products per slot within constraints → **snapshot** selections → validate (budget totals, specs, freshness, link validity, tag) → render → assemble board → log run + events. - **Output:** a budgeted shoppable board (per-slot product + price + live link), a styled render, a saved design (`run_id`). - **State (Supabase):** `runs` , `designs` , `slots` , `products` , `product_snapshots` , `click_events` , `user_inputs` , render references.

Why it matters. Without this line, prompts quietly absorb budget and link logic and the system becomes a beautiful board that's \$300 over budget with a dead link — the silent failure that destroys trust and earns nothing. This phase is where you protect money and trust the way Build B protected average integrity.

Deterministic vs LLM (the heart of it):

Responsibility	Owner
Interpret vibe/style from Q&A + photo	LLM, schema-constrained
Plan composition (which slots + budget split)	LLM proposes, code enforces totals/caps
Select products per slot within price band + spec rules	LLM over a candidate set, schema-constrained
Budget never exceeds target	Code only
Per-slot specs present (<code>bed_size</code> , <code>screen_size</code> , <code>dims</code>)	Rule + code
Price freshness within window before display	Code only
Link live + correct affiliate tag before display	Code only
Snapshot products/prices at generation time	Code only
Render styled image	Image model (presentation)
What the user actually sees	Deterministic assembler

What Claude Code should do. Produce `docs/system_map.md` , `docs/responsibility_split.md` , the core schemas (`slot` , `product` , `design` , `product_snapshot` , `click_event`), and a `docs/failure_modes.md` smell-test: what breaks if the slot taxonomy is weak, if state is hidden, if freshness isn't enforced, if tags aren't injected, if a design isn't snapshotted.

Common founder mistakes: comparability/budget rules in prompts; letting the model decide what's "on budget"; not defining state explicitly (later you can't tell if a design was logged, snapshotted, or had its links re-validated); treating the render as a source of product truth.

Success before moving on: a readable one-page system map; written schemas; a responsibility split; an explicit list of what's deterministic; and a clear answer to "how is a saved design reconstructable next week even though live prices changed?" (answer: the snapshot).

Phase 3 — Stack selection

What it is. The minimum viable stack, in your conventions.

Recommended MVP stack: - **Claude Code** — repo, implementation, tests, refactors, docs. - **Next.js/React** — consumer UI (intake → board → share). Funnel-critical; use the `frontend-design` skill (Phase 4F). - **Python + FastAPI** — engine, sourcing adapters, validators, API. - **Supabase (Postgres)** — designs, runs, products, snapshots, click events; row-level structure mirrors the pipeline. - **Railway** — deploy web, an async **worker** (render + sourcing), and **cron** (price/link refresh). Same trio you'd recognize from the furniture deploy. - **Image-generation API** — render, behind a swappable interface. - **Sourcing (phased):** v1 Amazon Associates + SiteStripe/curated; later a paid product-data API (Rainforest/Canopy-type) for programmatic catalog; later Temu via paid scraper API. - **Pydantic** — packet/contract validation. - **Click/event analytics** — first-class, because click-through *is* the v1 metric.

Deferred: PA-API (sales-gated), Temu data (no public API), operator/billing infra, spatial-layout/3D, vector search.

Why it matters. Stack choices get expensive once code exists. The seductive overbuild here is the render pipeline and a fancy frontend before the composition+sourcing+budget loop produces a board you'd buy from.

What Claude Code should do. Write `docs/stack_decision.md`, `docs/sourcing_strategy.md` (the Amazon → API → Temu sequence and the adapter contract that makes it a backend swap), `pyproject.toml`, `.env.example`, `docs/tooling_notes.md`, and `docs/deploy_railway.md` (web/worker/cron services, env vars, build commands).

Common founder mistakes: building render quality before cart quality; not isolating sourcing behind an interface (then Temu becomes a rewrite); putting the price/link refresh in the request path instead of a cron worker; skipping the worker and blocking the web request on render + sourcing.

Success before moving on: you can answer — what's the backend, the DB, how products are sourced, where async work runs, where cron runs, where click events live, and which tools are explicitly postponed.

Phase 4 — Repo creation

What it is. A clean repo mirroring the pipeline, with the deterministic layer as first-class citizens.

```
roomkit/
  README.md
  AGENTS.md
  CLAUDE.md           # if your Claude Code setup uses it; can mirror AGENTS.md
  .env.example
  pyproject.toml
  railway.json        # web / worker / cron service defs
  web/                # Next.js consumer app
    app/              # routes: intake, result board, share
    components/       # IntakeForm, BudgetSlider, StyleQuiz, RoomBoard, ProductCard,
ShareCard
  lib/api.ts          # typed client to FastAPI
  styles/
  app/
    main.py           # FastAPI entry
    api/routes.py     # POST /design, GET /design/{run_id}, POST /click
    deps.py
```

```

services/
  intake_service.py      # parse photo/dimensions + Q&A -> RoomRequest
  style_service.py       # RoomRequest -> StyleProfile (LLM)
  composition_service.py # StyleProfile + budget -> SlotPlan (LLM proposes; code enforces)
sourcing/
  base.py                # SourcingAdapter interface
  amazon_adapter.py      # v1 curated/SiteStripe; later product-data API
  temu_adapter.py        # later
  selection_service.py   # candidates per slot -> chosen products (LLM, constrained)
  snapshot_service.py    # freeze products/prices for a run
  render_service.py      # image generation (swappable)
  assembly_service.py    # deterministic final board
  refresh_worker.py      # locked cron: re-validate prices/links
schemas/
  room_taxonomy.py
  room_request.py
  style_profile.py
  slot.py
  slot_plan.py
  product.py
  product_snapshot.py
  design.py
  click_event.py
validators/
  budget_rules.py        # totals never exceed target
  spec_rules.py          # per-category required specs
  price_link_rules.py    # freshness window + live link + affiliate tag
  composition_rules.py   # required slots filled, no duplicate slots
prompts/
  interpret_style.md
  plan_composition.md
  select_products.md
  coding_agent/          # saved Claude Code prompts (Appendix D)
context/
  slot_taxonomy.yaml
  style_profiles.yaml
  category_spec_rules.yaml
  sourcing_policies.yaml
  freshness_policies.yaml
data/
  fixtures/              # snapshot fixtures for tests
  snapshots/             # runtime product/price snapshots (or in DB)
evals/
  composition_eval_set.csv
  coherence_rubric.md
tests/

```

```
test_intake.py
test_style.py
test_composition.py
test_budget_rules.py
test_spec_rules.py
test_price_link_rules.py
test_selection.py
test_snapshot.py
test_refresh_worker.py
test_assembly.py
test_api.py
docs/
  product_overview.md
  system_map.md
  sourcing_strategy.md
  deploy_railway.md
```

Why it matters. Claude Code pattern-matches the local codebase. Clean structure → safe extension; chaos → copied chaos. You can't write every line, but you can inspect this tree and ask: which folder owns budget enforcement, where do affiliate tags get injected, where are freshness checks, where do click events get logged, where does the locked refresh job live.

What Claude Code should do. After you approve the tree, scaffold folders + stub files + plain-English ownership comments, **no business logic yet**. Ask for the tree first, review, then the scaffold (use the Appendix D scaffold prompt).

Common founder mistakes: one giant file; routes/services/validators blurred; business rules in prompts; no `data/`, `evals/`, `context/` early (instructions then drift across chats); asking for full logic before the shape is approved.

Success before moving on: the tree mirrors the pipeline, every folder has one job, web and API entry points run, and you can explain each top-level folder.

Phase 4F — The frontend (the surface Build B never had)

What it is. The consumer UI is the funnel; conversion and shareability live here, so it is first-class, not an afterthought.

Why it matters. Your edge over render-first apps is the budgeted shoppable board — which only matters if the board is *legible and shareable*. A clumsy intake kills completion; an ugly board kills clicks and shares.

What Claude Code should do. Read the `frontend-design` skill before building UI. Build three screens: **(1) Intake** (photo upload or dimensions + a tight style/budget quiz — minimize fields; a human typing "12×14 bedroom, \$1,500, warm-minimalist" beats fragile blueprint OCR in v1), **(2) Result board** (the render up top for the hook, then the per-slot product cards with price + live buy button underneath, with a running budget total), **(3) Share** (a clean, screenshot-friendly card — the viral unit). Keep all data in React state / server; **no browser localStorage** (unsupported in this environment if you ever prototype in Artifacts).

Component set: `IntakeForm`, `StyleQuiz`, `BudgetSlider`, `RoomBoard`, `ProductCard` (image, name, price, "Buy on Amazon" with tag), `BudgetMeter`, `ShareCard`.

Common founder mistakes: over-building the frontend before the engine returns a good board; front-loading product images so it reads like an ad; burying the budget total; making the share output ugly.

Success before moving on: a user can go photo/dimensions → quiz → board → share in under a minute, the board shows render + budgeted cart, and the share card looks good enough to post.

Phase 5 — Core artifacts (with real contents)

These are system components, not docs. Create each as a real file. Below are strong first versions.

AGENTS.md (full)

```
# AGENTS.md

## Purpose
This repo powers RoomKit: a whole-room AI design and commerce engine.
It takes a room (photo or dimensions) plus style/budget inputs, plans a
composition of slots, selects real buyable products per slot within budget,
snapshots them, validates budget/specs/links/freshness deterministically,
renders a styled image, and presents a budgeted shoppable board.
Every design is a logged, idempotent run; every shown link earns affiliate revenue.

## Stack
- Frontend: Next.js/React in web/
- Backend: Python + FastAPI in app/ and services/
- DB/state: Supabase (Postgres)
- Deploy: Railway (web, worker, cron); source + CI on GitHub

## Main folders
- web/: consumer UI (intake, board, share) – funnel-critical
- services/: intake, style, composition, sourcing/, selection, snapshot, render, assembly,
refresh_worker
- schemas/: typed contracts (room request, style profile, slot, slot plan, product, snapshot,
design, click event)
- validators/: deterministic budget, spec, price/link, composition rules
- prompts/: LLM templates (style, composition, selection) + coding_agent/ saved prompts
- context/: slot taxonomy, style profiles, category spec rules, sourcing + freshness policies
- evals/: composition eval set + coherence rubric
- tests/: unit, validator, integration

## Core rules (always true)
- The slot taxonomy and category_spec_rules are authoritative.
- A board's product total MUST NOT exceed the target budget. Ever. This is code-enforced.
- Every product shown to a user MUST have: a validated live link, a price within the
  freshness window, and the correct affiliate tag in the buy URL.
- Each design is run-scoped: assign a run_id, log inputs/outputs, and make generation idempotent.
- Snapshot the exact products/prices used in a design at generation time. A saved design
```

```

    must be reconstructable later even if live prices/links have changed.
- Price/link freshness is enforced by code before display and before re-displaying an old design.
- The price/link refresh job runs in the cron worker under a lock; it must not double-run or
  deadlock.
- Business rules (budget, specs, links, tags, freshness) live in validators/, NEVER in prompts.
- Prompts may only interpret style, propose composition, and select within constraints.
- The render is presentation-only and is NEVER a source of product/price truth.
- Every product impression and every click is logged as data (this is the durable moat).

## Safe commands
- Run tests: `pytest`
- Lint/type: `ruff check .` / `mypy app services validators`
- Local API: `uvicorn app.main:app --reload`
- Local web: `cd web && npm run dev`

## Forbidden in tests / dev
- No writes that mutate a stored design's snapshot after creation.
- No live affiliate calls in tests; use fixtures.
- No skipping the budget/spec/link validators to "make it pass."

## How the agent must work
- Architecture-first: inspect and summarize before editing; propose a plan; wait for approval on
  anything touching budget, links, tags, snapshots, or the refresh lock.
- Build one stage at a time; do not jump ahead.
- After each change: explain changed files in plain English, state assumptions, add/extend tests,
  run them, and show failing evidence before patching.
- Update README, AGENTS.md, prompt templates, and context files when they should change.

```

context/slot_taxonomy.yaml

```

version: 1
country_scope: US
slots:
  - id: wall_art      {optional: true, default_budget_weight: 0.08, required_specs: []}
  - id: bedding       {optional: true, default_budget_weight: 0.10, required_specs: [bed_size]}
  - id: rug           {optional: true, default_budget_weight: 0.12, required_specs: [dimensions]}
  - id: lighting      {optional: true, default_budget_weight: 0.08, required_specs: []}
  - id: tv            {optional: true, default_budget_weight: 0.18, required_specs: [screen_size]}
  - id: sofa          {optional: true, default_budget_weight: 0.28, required_specs: []}
  - id: bed_frame     {optional: true, default_budget_weight: 0.22, required_specs: [bed_size]}
  - id: accent        {optional: true, default_budget_weight: 0.06, required_specs: []}
room_presets:
  bedroom: [bed_frame, bedding, rug, lighting, wall_art, accent]
  living_room: [sofa, rug, lighting, tv, wall_art, accent]
budget_rules:

```

```
total_must_not_exceed: target_budget
per_slot_overflow_flag: true
```

context/category_spec_rules.yaml

```
bedding:  {required: [bed_size], valid_bed_size: [twin, full, queen, king]}
bed_frame: {required: [bed_size], valid_bed_size: [twin, full, queen, king]}
tv:       {required: [screen_size]}
rug:      {required: [dimensions]}
wall_art: {required: []}
lighting: {required: []}
```

context/freshness_policies.yaml

```
price_freshness_hours: 24      # re-validate price if older than this before display
link_check_on_display: true    # verify link live before showing
refresh_cron: "0 */6 * * *"    # worker re-validates active designs' products every 6h
stale_design_warn_hours: 168    # warn/refresh a re-opened design older than a week
```

prompts/select_products.md (template shape)

Role: You are a product selector for a single room slot.

Input: slot id, target style profile, allowed price band, required specs, and a candidate list of real products (name, price, specs, source, buy_url).

Rules:

- Choose ONE product that best fits the style profile AND stays within the price band.
- The product MUST satisfy all required specs for this slot. If none in the candidate list do, return null and reason "no_spec_match".
- Do not invent products, prices, links, or specs. Only choose from the candidate list.
- Do not exceed the price band. Prefer the best style fit within band, not the cheapest.

Output (schema-constrained): product_id, fit_reason, confidence. No prose outside the contract.

validators (real)

```
def validate_budget(design, target_budget):
    total = sum(i.normalized_price for i in design.items)
    if total > target_budget:
        return False, "over_budget"
    return True, None

def validate_specs(item, slot, spec_rules):
    for spec in spec_rules[slot.id]["required"]:
        if not getattr(item, spec, None):
            return False, f"missing_{spec}"
    return True, None
```

```
def validate_price_link(item, freshness_hours, affiliate_tag):
    if item.price_age_hours > freshness_hours:
        return False, "stale_price"          # -> trigger refresh, do not show
    if not item.link_is_live:
        return False, "dead_link"
    if affiliate_tag not in item.buy_url:
        return False, "missing_affiliate_tag"
    return True, None

def validate_composition(design, preset_required_slots):
    present = {i.slot_id for i in design.items}
    missing = [s for s in preset_required_slots if s not in present]
    if missing:
        return False, f"missing_slots:{','.join(missing)}"
    return True, None
```

evals/composition_eval_set.csv (columns)

Column	Meaning
fixture_id	unique scenario id
room_type	bedroom / living_room
budget	target budget
style	named style profile
expected_required_slots	slots that must be filled
expected_on_budget	true
expected_all_specs_present	true
expected_all_links_valid	true
notes	why this case matters (e.g., tiny budget, oversized budget, no-keep items)

Seed scenarios: studio \$800 warm-minimalist · bedroom \$1,500 mid-century · living room \$1,500 cozy · bedroom \$400 (tiny-budget edge) · living room \$6,000 (oversized-budget edge) · bedroom "keeping the bed" (partial composition).

Why these artifacts matter / common mistakes / success: same logic as Build B — treat them as system components; keep AGENTS.md repo-specific; keep context in files not lucky prompt threads; never ship schemas without validators; build the eval set early before prompt drift. Success = a new teammate or agent could understand the system from README + AGENTS.md + context + prompts + schemas + validators + eval set.

Phase 6 — Claude Code usage model

Distinct modes, reasoning externalized into reviewable artifacts, never one vague "build this." Use **Plan Mode** for architecture and multi-file changes; scaffold with "no business logic yet"; implement smallest-slice-first; debug by isolating the failure class then smallest fix; test by generating + running + showing failures before patching; refactor by mapping ownership first. Keep AGENTS.md/context/prompts current; **require approval before any change touching budget, links, tags, snapshots, or the refresh lock**. Full prompt library in Appendix D.

Founder mistakes: treating Claude Code as the product author rather than implementer; one huge prompt; editing before summarizing the repo; no changed-file explanations; skipping tests because code "looks right"; letting prompt logic absorb business rules.

Phase 7 — Phased implementation roadmap

Each stage ends with tests green + artifact updates before the next begins. Exit criteria act as gates.

Stage	Build	Exit criteria
1	Repo scaffold (web + engine), docs, AGENTS.md, schemas, context, Railway services	Web + API run locally; folders exist; no business logic
2	Taxonomy/style/spec/freshness loaders	Context files load + validate
3	Intake → RoomRequest (photo/dimensions + Q&A)	Valid RoomRequest; missing fields null, not guessed
4	Style interpretation → StyleProfile (LLM)	StyleProfile validates against schema
5	Composition + budget allocation (code-enforced)	No SlotPlan exceeds budget; required slots present
6	Amazon sourcing adapter (curated/SiteStripe) + Product schema	Real products with price + live link + tag
7	Product selection per slot within constraints	Selections respect band + specs; null+reason if no match
8	Snapshot + deterministic validation (budget/spec/freshness/link/tag)	Invalid items blocked; design snapshotted + reconstructable
9	Render + board assembly + share UI	Coherent on-budget board, styled render, share card
10	Click tracking + run/design logging + analytics	Every impression, click, and run recorded
11	Refresh worker (locked cron) for price/link freshness	Stale prices/dead links caught off the request path; no double-run/deadlock
12	Hardening: evals, coherence rubric, fixtures, error paths	Loop runs repeatably; eval set green; failures reconstructable

Per-stage detail (real, not copy-paste): - **S3 intake:** the prompt/code/rule/manual split — LLM parses freeform Q&A into structured fields; code validates ranges (budget > 0, valid room_type); rule = required fields per room preset; manual = none. - **S5 composition:** LLM proposes slot weights; **code clamps** so $\Sigma \leq$ budget and re-

normalizes; rule = preset's required slots must appear; this is the single most important deterministic gate. - **S6 sourcing**: adapter returns candidates with price + buy_url + specs + tag; **the adapter interface is frozen here** so Amazon → product-data-API → Temu is a backend swap. - **S8 snapshot+validate**: snapshot first (freeze price/url/specs with timestamp), then validate against the snapshot; a saved design references its snapshot, never live data. - **S10 logging**: non-negotiable — this builds the moat; log impression + click with run_id, slot_id, product_id, style, budget, source. - **S11 refresh worker**: mirrors your advisory-lock work — single locked job, idempotent, updates snapshots' freshness, flips dead links, never runs in the request path.

Channel sequencing inside the build: S6 = Amazon curated/SiteStripe (ships now, no gate). At ~10 sales (PA-API) or sooner via paid product-data API, swap the adapter backend behind the frozen interface. Temu adapter post-loop via paid scraper API as opt-in budget mode.

Founder mistakes: too many stages per pass; skipping the snapshot (then designs aren't reconstructable and refresh corrupts them); putting refresh in the request path; deferring click logging; letting a later UI stage drive early architecture.

Phase 8 — Testing & validation

The product fails *quietly*: a board can look beautiful and be over budget, have a dead/untagged link, ship the wrong bed size, or reference live data instead of its snapshot. Test like Build B (your 316/316 instinct), green before each gate. - **Validator tests**: over-budget rejection; missing spec per category; stale price; dead link; missing/incorrect tag; missing required slot. - **Snapshot/idempotency tests**: same RoomRequest + run is idempotent; a stored design reads from its snapshot, not live data; re-running doesn't mutate an existing snapshot. - **Refresh-worker tests**: lock prevents double-run; idempotent; flips dead links; updates freshness without corrupting stored designs. - **Integration tests**: intake → ... → board happy path on fixtures; no live affiliate calls in tests. - **Eval loop**: the fixed scenarios checked every change for on-budget / required-slots / specs / live+tagged links. - **Coherence eval (the commercial one)**: rubric-scored small set — do the picks read as one room? Score manually first; this is what separates you from a random product list. *Would you buy from this board?* is the final gate.

Artifacts: tests/ , data/fixtures/ , evals/composition_eval_set.csv , evals/coherence_rubric.md , docs/testing_strategy.md , docs/logging_strategy.md .

Phase 9 — Pilot & launch

Bounded real run: bedroom + living room, a few styles, Amazon-only, a public link to 10–50 real users. Prove the loop, not the fantasy.

Metric	Tells you	Healthy
Design completion rate	Engine yields a full board	Most requests yield a complete, on-budget board
On-budget rate	Budget honesty	~100% (deterministic; any miss is a bug)
Link-validity rate	Links live + tagged	~100% before display
Click-through rate	Do people want the picks	The core signal
Share rate	Virality	Any meaningful sharing is strong
Coherence score	Are rooms good	Trending up

`docs/pilot_plan.md`, `docs/pilot_runbook.md`, `docs/review_checklist.md`, `scripts/run_pilot` (seeded scenarios + a few real users).

Launch decision: people click + share → validate loop, scale traffic, start operator-readiness. They don't → you learned it in a weekend's build + a pilot, not after building the company.

Phase 10 — Iteration & scale

Scale = more channels (product-data API → Temu budget mode), better coherence, more room types/styles, the **data layer maturing**, and the real inflection: **turning to operators** (subscription + sourcing spread) once a segment with proven WTP exists. Good scaling: swap Amazon adapter to programmatic catalog; add Temu; add operator accounts + billing; add spatial-layout reasoning as the v2 differentiator; build the intelligence/data product. Bad scaling: operator billing or 3D before the consumer loop converts; budget/link logic creeping into prompts; chasing render quality as the moat.

Appendix A — File-level implementation sequence

Scaffold + docs → schemas → validator stubs → context loaders → intake → style → composition (+budget enforcement) → Amazon adapter → selection → snapshot + validation → render + assembly + share UI → click logging → refresh worker → evals + hardening.

Appendix B — Milestone sequence

M1 scaffold/docs/schemas/AGENTS/context · M2 intake → style → composition with budget enforcement · M3 Amazon sourcing + selection returning real tagged products · M4 snapshot + full validation; reconstructable designs · M5 render + board + share; click logging live · M6 refresh worker + evals + pilot runbook.

Appendix C — Reference notes

Mirrors your Build B commitments: workflow-first, deterministic rules, run-scoping, snapshots, staleness guards, locked refresh, comprehensive tests, founder-first tool sequencing. New surfaces beyond Build B: frontend (Phase 4F), affiliate/transaction integrity (tags, link validity), render pipeline, and click/conversion data as the moat.

Appendix D — Copy-paste Claude Code prompt library

1. Repo understanding

Before changing anything, inspect this repo and explain in plain English:

- 1) what it currently does, 2) where intake, style, composition, selection, sourcing, snapshot, render, assembly, validation, and refresh live, 3) the safest insertion point for [feature], 4) which rules are deterministic vs prompt-driven, 5) which missing artifacts would make you more reliable here. Do not edit. End with a short plan.

2. Scaffold

Create the workflow-first RoomKit scaffold: Next.js web app in web/, FastAPI engine in app/+services/, typed schemas, validators (budget/spec/price_link/composition), context files, evals, tests, Railway service defs. Folders + stubs + plain-English ownership comments only. No business logic yet.

3. Composition (+budget enforcement)

Implement only composition planning: StyleProfile + budget -> SlotPlan. The LLM proposes slot weights; code MUST clamp and re-normalize so the total never exceeds budget, and MUST ensure the room preset's required slots are present. Add tests for over-budget rejection and missing required slots. Explain each file.

4. Amazon sourcing adapter

Implement only the Amazon adapter behind SourcingAdapter (base.py), using [curated list / product-data API]. Every product returned MUST carry normalized_price, a live buy_url, specs, and the affiliate tag. Freeze the interface. Add tests for missing price, dead link, missing tag, and no-spec-match. Explain assumptions.

5. Snapshot + validation

Implement product snapshotting and deterministic validation. Snapshot the chosen products/prices/urls with a timestamp BEFORE validation; a stored design must reference its snapshot, never live data. Then run budget, spec, freshness, link, and tag validators; block invalid items. Add idempotency tests (re-run does not mutate a snapshot) and tests that a stored design reads from its snapshot. Explain trade-offs.

6. Refresh worker

Implement the locked cron refresh worker: re-validate active designs' product prices/links on the freshness schedule, flip dead links, update freshness. It MUST run under a lock (no double-run), be idempotent, never run in the request path, and never corrupt a stored design's snapshot. Add tests for the lock, idempotency, and dead-link handling.

7. Debugging

Given this failing fixture and validator output, classify the failure: bad style interpretation, over-budget allocation, missing spec, stale price, dead link, missing tag, snapshot/idempotency, or lock/concurrency. Propose the smallest fix, name the files to change, and say why that layer is the right one.

8. Refactor

Map current ownership of composition, selection, snapshot, validation, and refresh. Propose the least disruptive refactor that keeps budget/link/tag/freshness logic OUT of prompts and inside validators. Do not implement until you explain what changes, why, what might break, and how to test it.

9. Test expansion

Extend tests for: over-budget board, missing bed_size, missing screen_size, stale price, dead link, missing tag, missing required slot, snapshot immutability, refresh-worker lock + idempotency. Run them; summarize failures; patch the smallest layer.

Appendix E — First-week command sequence

```
# Day 1: init + scaffold
mkdir roomkit && cd roomkit && git init && claude
/init
> Propose the repo tree (web + engine + validators + context + evals + tests + Railway services)
and the
> docs we need. Do not write code yet.

# Day 2: core artifacts
> Create README.md, AGENTS.md, .env.example, docs/system_map.md, docs/sourcing_strategy.md, and
starter
> schemas (room_request, style_profile, slot, slot_plan, product, product_snapshot, design,
click_event).
> No business logic. Explain each file.

# Day 3: intake + style
> Implement intake (photo/dimensions + Q&A -> RoomRequest, nulls not guesses) and style
interpretation.
> Add validator tests for budget range and required fields. Run and explain.

# Day 4-5: composition + sourcing
```

```

> Implement composition with code-enforced budget clamping, then the Amazon adapter behind the frozen
> SourcingAdapter interface returning real tagged products. Add tests. Show me which files own budget enforcement.

# Day 6-7: snapshot + validation + logging
> Implement snapshotting, the full deterministic validators, and click/run logging. Add idempotency tests.
> Confirm a stored design reads from its snapshot, not live data.

```

Appendix F — Pilot metrics & review checklist

Checklist before trusting outputs: every board's total $\leq$ budget with a visible total · every shown product has a live, correctly-tagged link · every required slot filled · each design references a snapshot and is reconstructable · stale prices/dead links are caught by the worker, not the user · the room reads as coherent · you would buy from it.

Appendix G — Founder review questions for every Claude Code change

Did it edit the right files or spread logic? · Preserve workflow-first? · Move any business rule (budget/link/tag/freshness) into a prompt that should be code? · Keep snapshot immutability and run idempotency intact? · Touch the refresh lock safely? · Add/extend tests and run them? · Explain trade-offs and unresolved edges? · Update README/AGENTS/prompts/context? · If this fails tomorrow, what artifacts reconstruct the failure?

Appendix H — Implementation risks & containment

Risk	Why dangerous	Containment
Budget logic in prompts	Boards silently exceed budget	Code-enforced clamping in composition + budget validator
Stale price between generation and click	User sees/buys at wrong price; trust + tag issues	Snapshot at design time + freshness window + locked refresh worker
Dead / untagged links	You do the work, earn nothing	Link-validity + tag validator before display
No snapshot	Saved designs not reconstructable; refresh corrupts them	Immutable per-run snapshot; designs read from snapshot
Refresh job double-run / deadlock	Corrupted freshness, races	Single locked idempotent cron worker (your advisory-lock pattern)
Adapter not isolated	Temu/API becomes a rewrite	Frozen SourcingAdapter interface at S6
Render treated as truth	Wrong product/price shown	Render is presentation-only; truth lives in snapshot + DB
Click logging skipped	No moat ever forms	S10 is non-negotiable; impressions + clicks logged with run_id

Risk	Why dangerous	Containment
Frontend built before engine	Polished funnel to a bad board	Engine returns a buy-worthy board before UI investment

Appendix I — The one-line throughline

A consumer-loved whole-room design-and-commerce app that, while delighting consumers and building an audience, quietly accumulates the proprietary furnishing-cost and style-demand data that becomes the operator product and the intelligence layer no competitor can copy by shipping a prettier render. Consumer is the wedge; operators + data are the business; build the loop first — with the same correctness discipline you already gave the furniture engine.